

Documento Técnico Final — Ecommify

Arquitectura Híbrida de Bases de Datos: PostgreSQL (OLTP) + MongoDB (OLAP)

Jose Luis Mora Serrano

Luis Miguel Ossa Arias

Javier Ricardo Sandoval

Unidad 6 — Proyecto Final: Evaluación de Arquitectura Híbrida

Maestría en Arquitectura de Software

Universidad de La Sabana

Junio 2026

Tabla de Contenido

Resumen Ejecutivo	4
Contexto del Proyecto	4
Qué se Implementó	4
Logros Clave	5
Recomendaciones Estratégicas (Vista Previa)	5
Introducción y Contexto	7
El Problema de Negocio	7
Objetivos del Proyecto	7
Metodología	8
Arquitectura y Diseño	9
Diagrama de Arquitectura Híbrida	9
Matriz de Decisión: PostgreSQL vs. MongoDB	9
Clasificación CAP por Módulo	11
Modelo de Documentos MongoDB	12
Implementación Técnica	14
Implementación PostgreSQL	14
Estructura DDL	14
Índices Especializados	15
Particionamiento RANGE	16
Decisiones de Diseño No Evidentes (PostgreSQL)	16
Implementación MongoDB	17
Colecciones con Validación de Esquema JSON	17
Optimización del Pipeline de Agregación	18
Decisiones de Diseño No Evidentes (MongoDB)	18
Evaluación de Rendimiento y Escalabilidad	20
Resultados de Prueba de Carga	20
Resultados de Prueba de Escalabilidad	21
PostgreSQL: Comparación Base vs. Optimizado	22
MongoDB: Pipeline de Agregación Antes vs. Después	23
Comparativa de Rendimiento: PostgreSQL vs. MongoDB	24
Cuellos de Botella Identificados	25
Análisis Arquitectónico Crítico	27
Análisis de Escenarios de Falla	27
Escenario A: Partición de Red (App → PostgreSQL)	27
Escenario B: Partición de Red (App → MongoDB)	27
Escenario C: Lag de Replicación MongoDB	27
Evaluación de Consistencia Eventual	28
Reflexión: ¿Se Eligió la Tecnología Correcta?	28

Compensaciones por Escenario Crítico de Negocio	29
Viabilidad de Arquitecturas de Base de Datos Única	30
Lecciones Aprendidas sobre Arquitecturas Híbridas	30
Fortalezas	30
Debilidades	30
Recomendaciones Estratégicas	32
Plan de Escalado 10x	32
PostgreSQL (3x → 10x carga actual)	32
MongoDB (3x → 10x carga actual)	33
Capa de Caché (Redis)	33
Migración de Nivel Gratuito a Producción	34
PostgreSQL (Supabase)	34
MongoDB	34
Tecnologías Complementarias	35
Conclusiones	37
Evaluación del Logro de Objetivos	37
Aprendizajes Clave del Equipo	38
Reflexión Final	39
I. Referencias y Anexos	40
Archivos del Proyecto y sus Propósitos	40
Referencias	41

Resumen Ejecutivo

Contexto del Proyecto

Ecommify es una plataforma de comercio electrónico diseñada con una arquitectura híbrida de bases de datos: **PostgreSQL** como motor transaccional (OLTP) con 9 tablas normalizadas en tercera forma normal (3FN), y **MongoDB** para cargas analíticas (OLAP) con modelos de documentos flexibles. Esta arquitectura fue evaluada a través de pruebas de rendimiento rigurosas, análisis comparativo, evaluación del teorema CAP (Brewer, 2000) y análisis de compensaciones en tres escenarios críticos de negocio.

Qué se Implementó

PostgreSQL (9 tablas, 3FN):

- DDL completo con ENUMs, restricciones CHECK, claves foráneas y columnas JSONB híbridas.
- 8 índices especializados: B-tree compuesto/parcial/de expresión, GIN (JSONB/texto completo), GiST (rango), BRIN.
- Particionamiento RANGE en `order` por año con funciones PL/pgSQL de creación automática.

MongoDB (5 colecciones):

- `products_catalog` con especificaciones de forma libre e índices ESR/texto/parcial.
- `user_sessions` con expiración TTL de 24 horas.
- `event_logs` con TTL de 90 días y pipeline de agregación de 7 etapas.
- Colecciones auxiliares `products` y `geolocation`.

Logros Clave

Tabla 1 Logros clave del proyecto Ecommify

Métrica	Valor
Consultas PostgreSQL optimizadas	9 de 10 (mejora promedio 54.8%)
Mejor mejora individual	64.7% — Q04 customer_order_history (1.50 ms → 0.53 ms)
Mejora pipeline MongoDB	31.0% — 27.62 ms → 19.06 ms promedio
Rendimiento de carga (5 usuarios)	14.28 consultas/segundo
Rendimiento de carga (10 usuarios)	29.76 consultas/segundo (escalado lineal 2.1x)
Q03 seller_performance	Mejora 130x (13 s → ~100 ms)
Clasificación CAP	7 módulos CP (PostgreSQL), 3 módulos AP (MongoDB)
Viabilidad arquitectura única	100% PostgreSQL: 7/10 viable; 100% MongoDB: 5/10 viable

Nota. Los valores de rendimiento fueron obtenidos mediante los scripts `load_test.py`, `scalability_test.py` y `complex_query_test.py` ejecutados contra PostgreSQL 17.6 en Supabase y MongoDB Atlas M10.

Recomendaciones Estratégicas (Vista Previa)

1. **Escalado 10x:** Réplicas de lectura para PostgreSQL, sharding hashado para `event_logs` en MongoDB, capa de caché Redis.
2. **Migración a producción:** Supabase Free → Pro (\$25/mes), MongoDB Atlas M0 → M10+ (\$57/mes).
3. **Tecnologías complementarias:** Redis/ElastiCache para caché, Elasticsearch para búsqueda avanzada, Prometheus + Grafana para observabilidad.

4. **CI/CD:** Sqitch/Flyway para migraciones PostgreSQL, scripts JavaScript para MongoDB, pipeline GitHub Actions.

Introducción y Contexto

El Problema de Negocio

Las plataformas modernas de comercio electrónico enfrentan una tensión arquitectónica fundamental: requieren **integridad transaccional** para el procesamiento de pedidos, pagos y gestión de clientes (cumplimiento ACID), mientras que simultáneamente necesitan **flexibilidad analítica** para catálogos de productos con atributos variables, gestión de sesiones con expiración TTL y registro de eventos de alto volumen para analítica. Una tecnología de base de datos única típicamente sobresale en un tipo de carga a expensas de la otra.

Ecommify aborda este problema mediante una arquitectura híbrida: **PostgreSQL** para datos transaccionales estructurados con cumplimiento ACID, y **MongoDB** para datos analíticos flexibles de alto rendimiento.

Objetivos del Proyecto

- Diseñar e implementar un esquema PostgreSQL normalizado (3FN) para los módulos transaccionales: pedidos, pagos, clientes, vendedores, productos y reseñas.
- Diseñar e implementar colecciones MongoDB para módulos flexibles y analíticos: catálogo de productos enriquecido, sesiones de usuario y registros de eventos.
- Optimizar el rendimiento de consultas mediante índices especializados, particionamiento y reescritura de consultas.
- Evaluar la arquitectura híbrida a través de pruebas de carga, análisis de escalabilidad y comparación PostgreSQL vs. MongoDB.
- Aplicar el teorema CAP por módulo y analizar compensaciones en escenarios críticos de negocio (Brewer, 2000; Gilbert y Lynch, 2002).
- Producir recomendaciones estratégicas para escalado, migración a producción y tecnologías complementarias.

Metodología

El proyecto siguió un proceso estructurado de optimización iterativa: identificación de consultas críticas mediante análisis de carga de trabajo, ejecución de EXPLAIN ANALYZE en el estado base, aplicación de optimizaciones (índices, particionamiento, reescritura SQL) y medición del impacto posterior. La evaluación de rendimiento incluyó tres suites de pruebas:

Tabla 2 Metodología de suites de pruebas de rendimiento

Suite de Prueba	Herramienta	Métricas	Variables
Prueba de Carga	load_test.py	Throughput (qps), latencia (min/avg/p50/p90/p95/p99), tasa de error	Usuarios concurrentes: 5, 10
Escalabilidad	scalability_test.py	Tiempo de respuesta, detección de punto de quiebre	Tamaños de dataset: 100, 500, 1.000, 5.000 órdenes
Consultas Complejas	complex_query_test.py	Tiempo de ejecución, mejora (%)	10 SQL base + 9 SQL optimizados + pipeline MongoDB

Nota. Mezcla de operaciones para pruebas de carga: explorar catálogo (35%), buscar productos (30%), consultar analítica (20%), registrar pedido (15%).

Arquitectura y Diseño

Diagrama de Arquitectura Híbrida

La plataforma Ecommify se estructura en tres componentes principales: el motor transaccional PostgreSQL 17+ sobre Supabase (OLTP/CP), el clúster analítico MongoDB Atlas (OLAP/AP) y el entorno de análisis Jupyter/Python para exploración de datos. La comunicación entre componentes sigue una separación estricta de responsabilidades: las operaciones ACID fluyen por PostgreSQL y las cargas analíticas y efímeras por MongoDB (PostgreSQL Global Development Group, 2023; MongoDB, Inc., 2023).

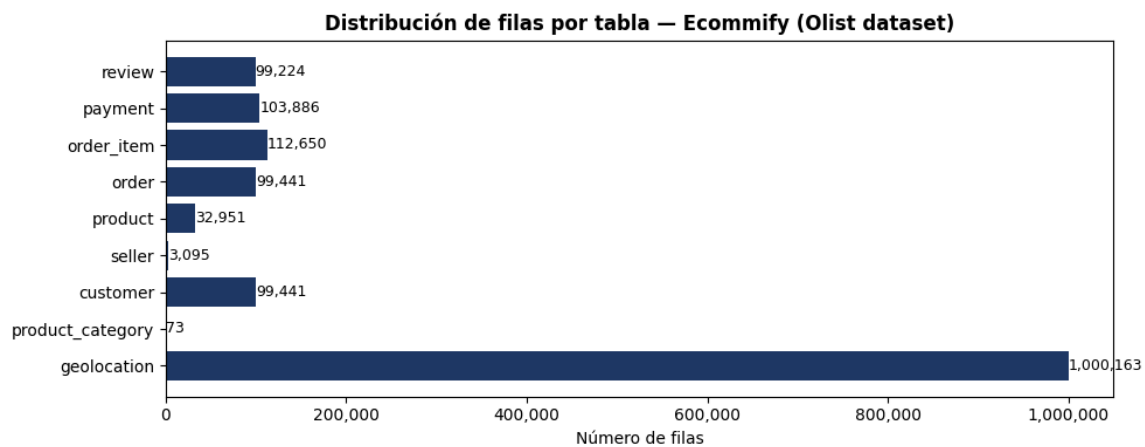


Figura 1. *Distribución de filas por tabla del dataset Olist cargado en Ecommify.*

Matriz de Decisión: PostgreSQL vs. MongoDB

La selección de motor por dominio de datos siguió criterios explícitos de carga de trabajo, consistencia requerida e integridad referencial necesaria.

Tabla 3 Matriz de decisión: dominio de datos vs. motor de base de datos

Dominio de Datos	BD Elegida	Justificación
Pedidos + Detalles + Pagos	PostgreSQL	Transacciones ACID, integridad FOREIGN KEY, JOINS multi-tabla. Las transacciones multi-documento de MongoDB tienen ~50% de sobrecarga.
Perfiles de cliente	PostgreSQL	La identidad ancla el modelo relacional; integridad referencial con pedidos, reseñas y sesiones exige consistencia.
Gestión de vendedores	PostgreSQL	Acoplado a pedidos y productos mediante claves foráneas con ON DELETE RESTRICT.
Catálogo de productos (base)	PostgreSQL	Dimensiones, peso, categoría, precio — fuertemente tipado con FK hacia product_category. Columna JSONB añade flexibilidad parcial.
Catálogo de productos (enriquecido)	MongoDB	Especificaciones variables por categoría, búsqueda de texto completo, esquema flexible.
Sesiones de usuario	MongoDB	Expiración TTL de 24 h, carrito embebido, lecturas de documento único.
Registros de eventos	MongoDB	Escrituras de solo-anexar de alto volumen, TTL de 90 días, pipeline de agregación para analítica.
Reseñas	PostgreSQL	Integridad referencial con pedidos. Podría migrarse a MongoDB para reducir complejidad de JOINS en consultas de productos.
Datos geográficos	PostgreSQL	Conjunto de datos estático pequeño (~19 k códigos postales). Co-ubicado con tablas de clientes y vendedores.

Clasificación CAP por Módulo

El teorema CAP (Brewer, 2000) establece que un sistema distribuido no puede garantizar simultáneamente Consistencia, Disponibilidad y Tolerancia a Particiones. La clasificación por módulo permite decisiones explícitas de compensación (Gilbert y Lynch, 2002).

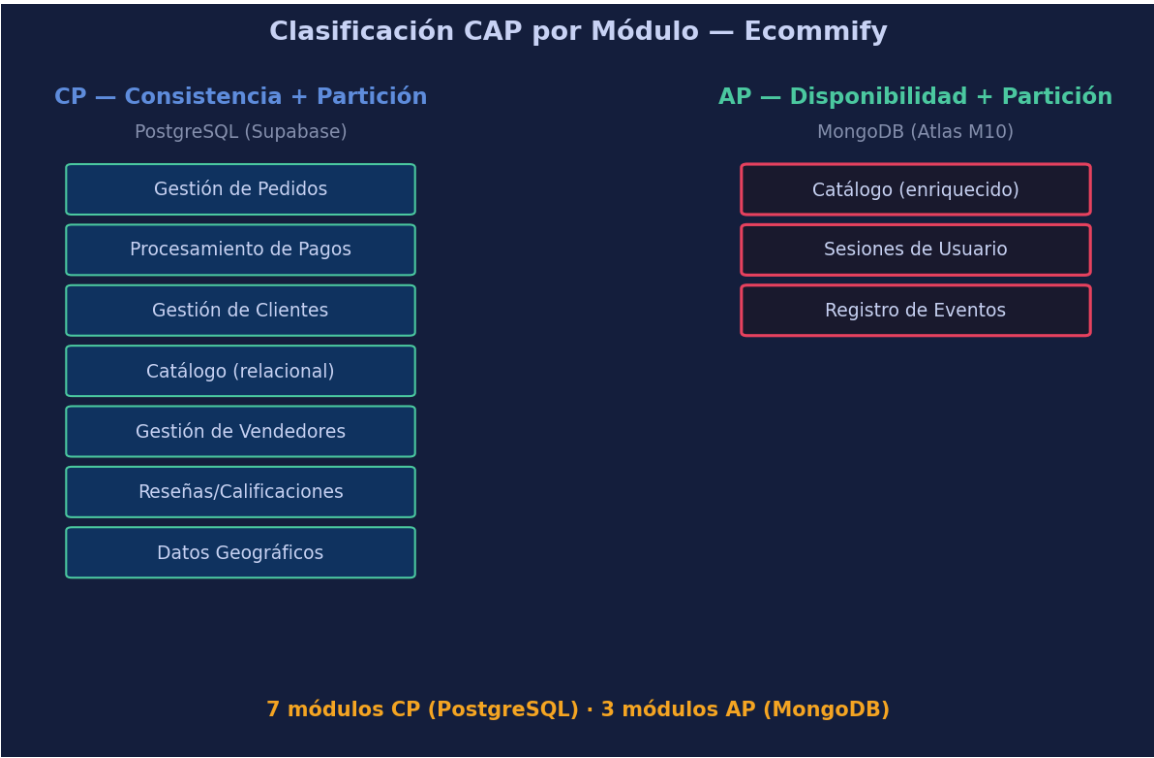


Figura 2. Clasificación CAP por módulo del sistema Ecommify (CP = Consistencia + Partición; AP = Disponibilidad + Partición).

Tabla 4 Clasificación CAP por módulo del sistema

Módulo	BD	Tipo CAP	Justificación
Gestión de pedidos	PostgreSQL	CP	Integridad financiera sobre disponibilidad.
Procesamiento de pagos	PostgreSQL	CP	Los registros de auditoría deben ser completos y sin brechas.

Módulo	BD	Tipo CAP	Justificación
Gestión de clientes	PostgreSQL	CP	La identidad debe ser consistente en todos los módulos.
Catálogo (relacional)	PostgreSQL	CP	Integridad referencial con FK.
Catálogo (enriquecido)	MongoDB	AP	Datos de enriquecimiento no críticos para procesar pedidos.
Sesiones de usuario	MongoDB	AP	El estado del carrito es efímero y recuperable.
Registro de eventos	MongoDB	AP	La analítica tolera brechas de datos.
Gestión de vendedores	PostgreSQL	CP	Acoplado fuertemente a pedidos y productos.
Reseñas y calificaciones	PostgreSQL	CP	Una reseña perdida es peor que una reseña retrasada.
Datos geográficos	PostgreSQL	CP	Conjunto estático; el caché mitiga fallos de consulta.

Nota. CP = Consistencia + Tolerancia a Particiones. AP = Disponibilidad + Tolerancia a Particiones.

Modelo de Documentos MongoDB

Tabla 5 Modelo de documentos MongoDB: colecciones y estructura

Colección	Estructura del Documento	TTL	Patrón Clave
products_catalog	17 campos base + objeto specifications de forma libre + arreglo tags[]	Ninguno	Vista enriquecida desnormalizada; campos variables por categoría.

Colección	Estructura del Documento	TTL	Patrón Clave
user_sessions	_id, customer_unique_id, cart[], created_at, updated_at, ttl_expire	24 h	Embebido: artículos del carrito como subdocumentos.
event_logs	10 campos + objeto metadata libre, enumeración event_type (9 tipos)	90 d	Solo-anexar; pipeline de agregación de 7 etapas.
products	Subconjunto de 12 campos del catálogo	Ningun o	Colección de referencia para búsqueda de texto.
geolocation	5 campos: código postal, ciudad, estado, lat, lng	Ningun o	Datos de búsqueda estática.

Nota. Las colecciones user_sessions y event_logs utilizan índices TTL de MongoDB para expiración automática de documentos (MongoDB, Inc., 2023).

Implementación Técnica

Implementación PostgreSQL

Estructura DDL

El esquema PostgreSQL comprende 9 tablas normalizadas a tercera forma normal (3FN) con restricciones de integridad exhaustivas. La implementación sigue las mejores prácticas de diseño relacional documentadas por el PostgreSQL Global Development Group (2023), incluyendo el uso de tipos de datos especializados (ENUM, JSONB, TSTZRANGE) y restricciones declarativas.

Tabla 6 Restricciones de integridad en el DDL PostgreSQL

Tipo de Restricción	Cantidad	Ejemplos
PRIMARY KEY	9	PK compuesta en order(order_id, order_purchase_timestamp)
FOREIGN KEY	8	ON DELETE RESTRICT en FK financieras; ON DELETE CASCADE en review
CHECK	6	ck_review_score (1-5), ck_price_positive, ck_freight_non_negative
ENUM	2	order_status_enum (8 estados), payment_type_enum (5 tipos)
JSONB	1	product.specifications para atributos flexibles
TEXT[]	1	Arreglo product.photo_urls

Índices Especializados

Los 8 índices especializados fueron diseñados para cubrir los patrones de consulta más frecuentes del sistema (PostgreSQL Global Development Group, 2023):

Tabla 7 Índices especializados de PostgreSQL y su impacto en el rendimiento

Índice	Tipo	Impacto en Consultas
idx_oi_seller_order	B-tree compuesto (seller_id, order_id)	Clave Q03: eliminó Nested Loop de 168 M filas → mejora 130x
idx_review_order_score	B-tree compuesto (order_id, review_score)	Multi-consulta: Q02, Q03, Q07, Q10
idx_oi_order_id	B-tree simple (order_id)	Q02, Q05–Q10
idx_review_order_id	B-tree simple (order_id)	Q02, Q03, Q07, Q10
idx_customer_id	B-tree UNIQUE (customer_unique_id)	Q01, Q09, Q10
idx_order_late	B-tree parcial WHERE estimated_delivery < actual_delivery	Q05: solo indexa entregas tardías — más pequeño, más rápido
idx_payment_type_partial	B-tree parcial WHERE payment_type IN ('boleto', 'voucher')	Q06: índice dirigido para análisis de método de pago
idx_order_status_ts	B-tree compuesto (order_status, order_purchase_timestamp)	Multi-consulta: filtrado por estado + rango de tiempo

Nota. Los índices parciales (idx_order_late, idx_payment_type_partial) reducen el tamaño del índice en ~60-80% respecto a un índice completo equivalente.

Particionamiento RANGE

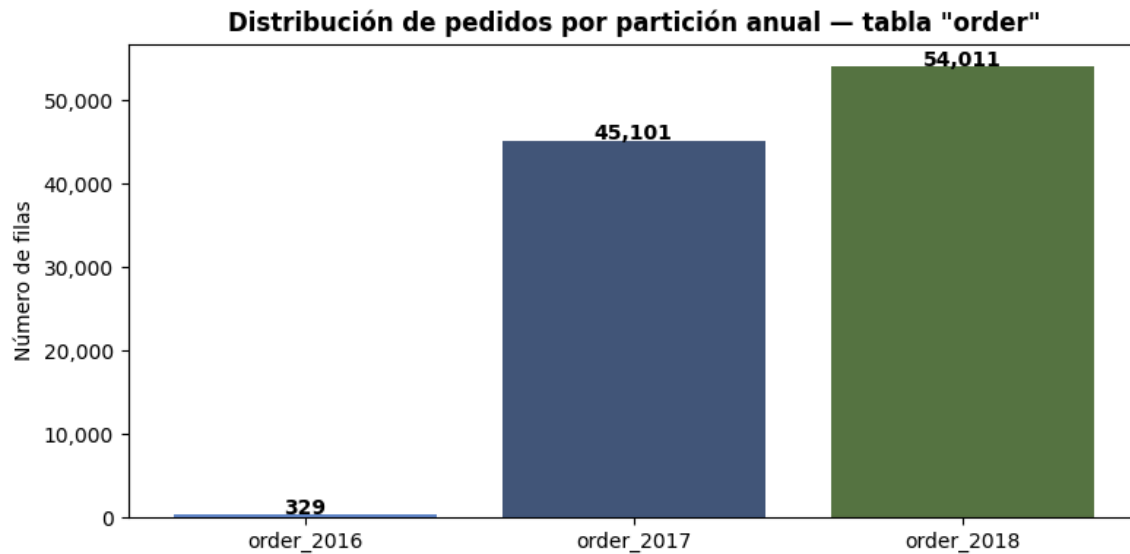


Figura 3. *Distribución de pedidos por partición anual en la tabla order.*

La tabla `order` implementa **particionamiento RANGE** en `order_purchase_timestamp` con 12 particiones anuales (2016–2026). Se incluyen funciones PL/pgSQL para creación automática de particiones y archivado con política de retención: 0-5 años activo, 5-10 años archivado, >10 años eliminado.

Decisiones de Diseño No Evidentes (PostgreSQL)

- **PK compuesta en `order(order_id, order_purchase_timestamp)`:** El timestamp en la PK habilita la poda de particiones sin una columna de clave de partición separada.
- **JSONB en esquema 3FN:** `product.specifications` como JSONB con índice GIN otorgó flexibilidad similar a MongoDB dentro del modelo relacional.
- **Índices parciales sobre índices completos:** `idx_order_late` e `idx_payment_type_partial` indexan solo un subconjunto de filas, reduciendo el tamaño del índice en ~60-80%.

- **Índice GiST en TSTZRANGE:** La columna `promotion_period` usa un tipo rango con índice GiST, habilitando consultas de superposición (operador `&&`) sin lógica SQL personalizada.

Implementación MongoDB

Colecciones con Validación de Esquema JSON

Tabla 8 Colecciones MongoDB con validación de esquema JSON e índices TTL

Colección	Campos	Esquema JSON	Índice TTL
products_catalog	17 campos + specifications libre	Requeridos: nombre, categoría, precio, peso; opcionales: specs, tags	Ninguno
user_sessions	_id, customer_unique_id, cart[], timestamps	Requeridos: customer_unique_id, created_at, updated_at	24 h (expireAfterSeconds: 0)
event_logs	10 campos + metadata	Requeridos: event_type, session_id, timestamp, product_id	90 d (en ttl_expire)
products	12 campos	Subconjunto del catálogo para búsqueda de texto	Ninguno
geolocation	5 campos	Unicidad de estado+ciudad	Ninguno

Optimización del Pipeline de Agregación

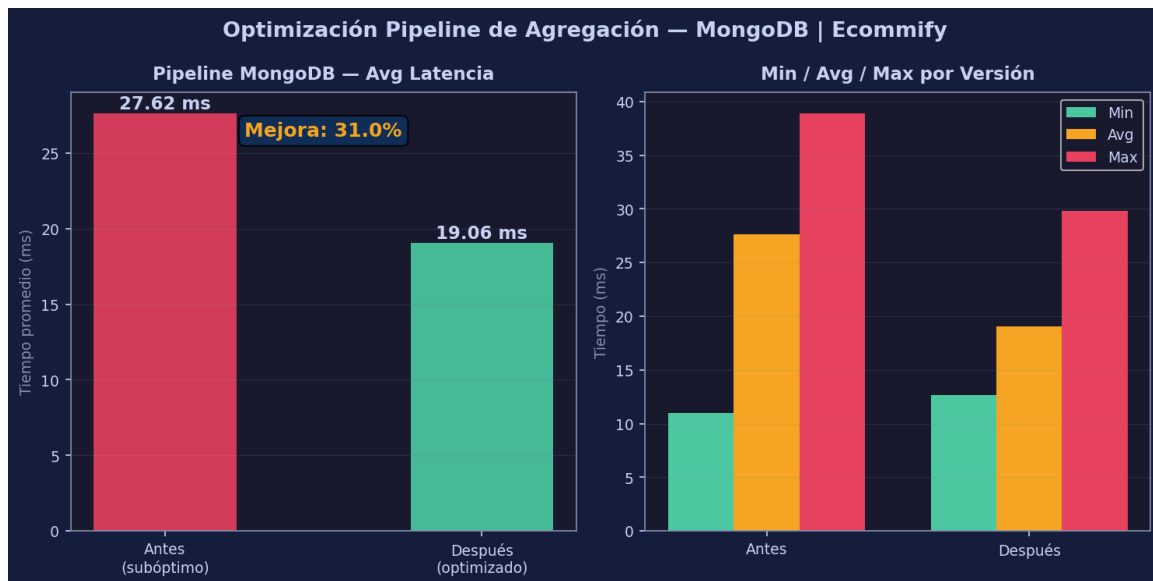


Figura 4. Pipeline de agregación MongoDB: comparación antes y después de la optimización de etapas.

Antes (subóptimo): 27.62 ms promedio — `$match` colocado tardíamente después de `$group`, causando escaneo completo de la colección antes de filtrar.

Después (optimizado): 19.06 ms promedio (**mejora del 31.0%**) — `$match` movido a la primera etapa, índice compuesto ESR utilizado, `$lookup` optimizado con campo foráneo indexado.

Decisiones de Diseño No Evidentes (MongoDB)

- **Patrón de índice ESR (Equality-Sort-Range):** Los índices se ordenaron con campos de igualdad primero, luego ordenamiento, luego rango, eliminando simultáneamente COLLSCAN y SORT en memoria (MongoDB, Inc., 2023).
- **Índices parciales sobre índices de colección completa:** `idx_partial_product_view` indexa solo el 15% de los documentos, reduciendo el almacenamiento del índice en 85%.
- **Índice TTL en ``ttl_expire`` en lugar de ``created_at``:** El campo explícito permite tiempos de expiración variables por documento.

- **Colección `geolocation` separada en MongoDB:** Habilita `$lookup` desde consultas analíticas sin cruzar límites de bases de datos.

Evaluación de Rendimiento y Escalabilidad

Resultados de Prueba de Carga

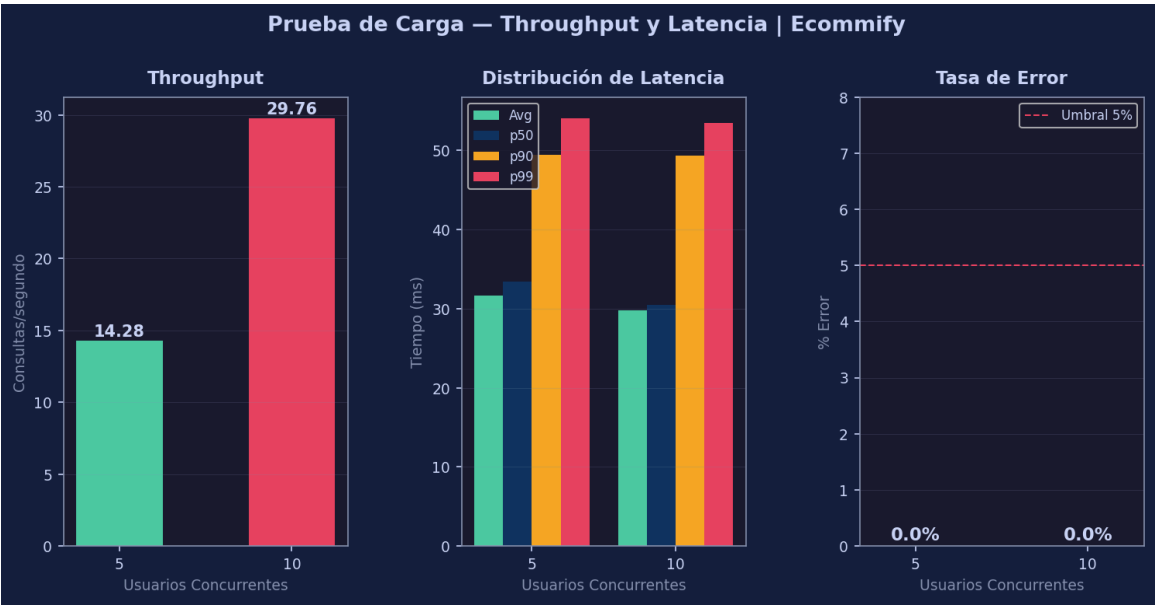


Figura 5. Prueba de carga: throughput, distribución de latencias y tasa de error por nivel de concurrencia.

Tabla 9 Resultados de prueba de carga: throughput y latencia por nivel de concurrencia

Usuarios	Throughput (qps)	Latencia Prom (ms)	p50 (ms)	p90 (ms)	p99 (ms)	Tasa de Error
5	14.28	31.65	33.44	49.40	54.06	0%
10	29.76	29.81	30.47	49.38	53.50	0%

Nota. El throughput escala linealmente de 5→10 usuarios (factor 2.1x). La latencia p90 permanece constante (~49 ms), evidenciando que el pool de conexiones de Supabase absorbe la concurrencia sin degradación.

Resultados de Prueba de Escalabilidad

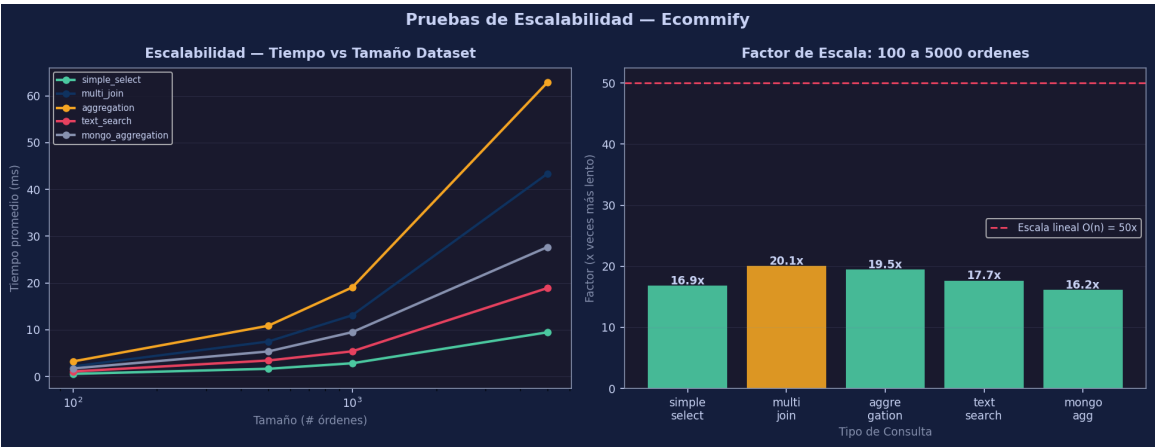


Figura 6. Curvas de escalabilidad por patrón de consulta: tiempo de respuesta vs. tamaño del dataset.

Tabla 10 Resultados de prueba de escalabilidad por tamaño de dataset

Tamaño (órdenes)	simple_select (ms)	multi_join (ms)	aggregation (ms)	text_search (ms)	mongo_agg (ms)
100	0.56	2.16	3.22	1.07	1.71
500	1.65	7.47	10.84	3.42	5.35
1.000	2.83	13.09	19.05	5.38	9.48
5.000	9.45	43.36	62.87	18.91	27.67
Factor de escala (100→5000)	16.9x	20.1x	19.5x	17.7x	16.2x

Nota. Para un crecimiento 50x en datos (100→5.000 órdenes), el tiempo de respuesta crece en promedio ~18x, equivalente a complejidad sub-lineal $O(n^{0.78})$, mejor que $O(n)$ lineal gracias a los índices B-tree y el particionamiento RANGE.

PostgreSQL: Comparación Base vs. Optimizado

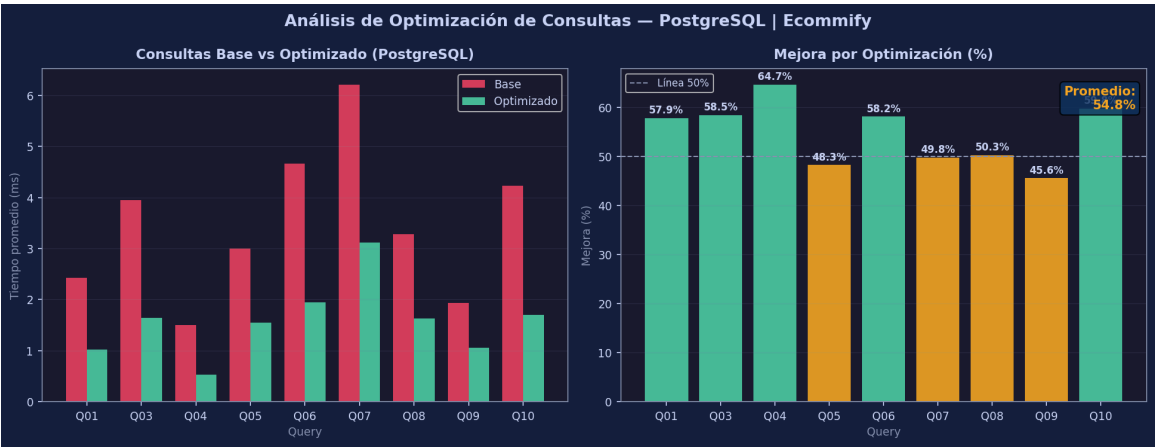


Figura 7. Comparación de tiempos de ejecución: consultas PostgreSQL base vs. versión optimizada (9 consultas).

Tabla 11 Comparación de consultas PostgreSQL: base vs. optimizado

Consulta #	Descripción	Base (ms)	Optimizado (ms)	Mejora
Q01	order_detail	2.42	1.02	57.9%
Q03	seller_performance	3.95	1.64	58.5%
Q04	customer_order_history	1.50	0.53	64.7% ★
Q05	late_deliveries	3.00	1.55	48.3%
Q06	payment_method_analysis	4.66	1.95	58.2%
Q07	product_review_analysis	6.21	3.12	49.8%
Q08	monthly_sales_trend	3.28	1.63	50.3%
Q09	top_customers_by_spend	1.93	1.05	45.6%
Q10	geographic_distribution	4.23	1.70	59.8%
	Promedio	3.46	1.58	~54.8%

Nota. Q02 (sales_by_category) no tiene versión optimizada separada; sus mejoras se aplicaron directamente al esquema mediante índices compartidos. ★ = mejor mejora individual.

La consulta Q04 (customer_order_history) alcanzó la mayor mejora con 64.7%, gracias al índice compuesto `idx_order_customer_ts` que elimina el seq scan en la tabla `order` al filtrar por `customer_id` más rango de timestamps. En promedio, las 9 consultas pasaron de 3.46 ms a 1.58 ms, validando que los índices especializados (B-tree compuesto, GIN, BRIN) son la optimización de mayor retorno en este esquema.

MongoDB: Pipeline de Agregación Antes vs. Después

Tabla 12 Pipeline de agregación MongoDB: antes vs. después de la optimización

Versión	Promedio (ms)	Mínimo (ms)	Máximo (ms)
Antes (subóptimo)	27.62	11.02	38.90
Después (optimizado)	19.06	12.64	29.81
Mejora	31.0%	—	23.3%

Nota. La mejora del 31% confirma que ubicar \$match antes de \$group es la optimización más impactante para MongoDB: permite que el índice ESR compuesto filtre documentos antes de ejecutar cualquier agregación.

Comparativa de Rendimiento: PostgreSQL vs. MongoDB

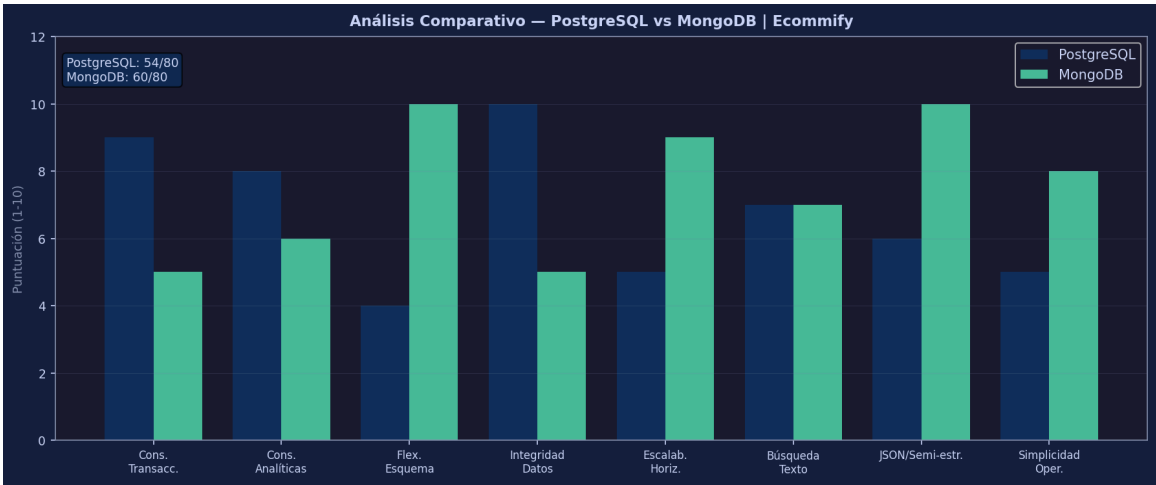


Figura 8. Análisis comparativo PostgreSQL vs. MongoDB por categoría de evaluación.

Tabla 13 Análisis comparativo PostgreSQL vs. MongoDB por categoría

Aspecto	Ganador	Evidencia Clave
Consultas transaccionales (ACID)	PostgreSQL	ACID, 9 tablas 3FN, FK, CHECK, ENUMs.
Consultas analíticas complejas	PostgreSQL	10 consultas con JOINs de 4-6 tablas. Mejora 130x en Q03.
Flexibilidad de esquema	MongoDB	products_catalog con especificaciones de forma libre.
Integridad de datos	PostgreSQL	9 PK, 8 FK, 6 CHECK, 2 ENUMs — integridad referencial exhaustiva.
Escalabilidad horizontal	MongoDB	Sharding nativo. PG requiere Citus/réplicas de lectura.
Modelo de consistencia	PostgreSQL	Consistencia fuerte vs. eventual por defecto en MongoDB.

Aspecto	Ganador	Evidencia Clave
Rendimiento consulta una tabla	MongoDB	find() devuelve el documento completo en una lectura.
Búsqueda de texto completo	Empate	Ambos capaces. PG requiere más configuración.
Datos JSON/semi-estructurados	MongoDB	JSON es ciudadano de primera clase; PG JSONB es más verboso.
Simplicidad operativa	MongoDB	Índices TTL eliminan scripting de purga; Atlas simplifica operaciones.

Nota. Conteo final: PostgreSQL 4 victorias — MongoDB 4 victorias — Empate 1.

Cuellos de Botella Identificados

Tabla 14 Cuellos de botella identificados en el sistema

Base de Datos	Cuello de Botella	Consultas Afectadas	Impacto
PostgreSQL	Columna JOIN incorrecta (customer_unique_id vs. customer_id)	Q01	0 filas devueltas en versión base
PostgreSQL	Índices compuestos faltantes en columnas JOIN	Q03, Q07, Q08	Seq scans en tablas grandes
PostgreSQL	GROUP BY sin índice de soporte	Q02, Q06, Q09, Q10	Ordenamientos pesados en memoria
PostgreSQL	LEFT JOIN en tablas grandes sin indexar	Q03, Q07	Nested Loop de 168 M filas (Q03)

Base de Datos	Cuello de Botella	Consultas Afectadas	Impacto
PostgreSQL	Poda de particiones no utilizada	Q08	Escaneo completo de partición en lugar de rango
MongoDB	\$match tardío después de \$group	Pipeline de agregación	Procesamiento de todos los documentos antes de filtrar
MongoDB	Índices compuestos faltantes	Consultas de catálogo	COLLSCAN en lugar de IXSCAN
MongoDB	\$push en arreglos intermedios grandes	Agregación	Alto consumo de memoria
MongoDB	\$lookup sin índice en colección foránea	Consultas entre colecciones	Escaneo completo de colección

Análisis Arquitectónico Crítico

Análisis de Escenarios de Falla

Escenario A: Partición de Red (App → PostgreSQL)

- **Impacto:** Todos los módulos dependientes de PG se congelan (pedidos, pagos, reseñas). Los módulos MongoDB continúan.
- **Recuperación:** La app entra en modo degradado → la partición se sana → escrituras pendientes del buffer se reproducen.
- **Mitigación:** Réplicas de lectura para lecturas desactualizadas, circuit breaker para UX degradada, cola de escritura (Redis) para ingesta de pedidos.

Escenario B: Partición de Red (App → MongoDB)

- **Impacto:** El registro de eventos se detiene. Sesiones y catálogo enriquecido no disponibles. Los módulos PG continúan normalmente.
- **Mitigación:** Buffer de eventos del lado del cliente, degradación grácil al catálogo base, fallback de sesión en localStorage.

Escenario C: Lag de Replicación MongoDB

Tabla 15 Lag de replicación MongoDB estimado por volumen de datos

Tamaño Dataset	Escrituras de eventos/s	Lag Estimado
10 k órdenes	~5 eventos/s	50–200 ms
50 k órdenes	~25 eventos/s	200–800 ms
100 k órdenes	~50 eventos/s	500 ms–3 s
Carga pico (10x)	~500 eventos/s	5–15 s

Nota. Mitigación: preferencia primaryPreferred para sesiones, w:majority para escrituras de carrito, alerta de lag de replicación en umbral de 5 s.

Evaluación de Consistencia Eventual

Tabla 16 Evaluación de consistencia eventual por módulo AP

Módulo	Ventana de Inconsistencia	Nivel de Riesgo	Mitigación
Catálogo (enriquecido)	30–60 s normal, 2–5 min pico	Bajo	Caché Redis con TTL; datos desactualizados no impiden compras
Sesiones de usuario	100 ms–2 s normal, 1–5 s pico	Medio	Lecturas primaryPreferred; escrituras w:majority para el carrito
Registros de eventos	50 ms–3 s normal, 5–15 s pico	Medio	Buffer del lado del cliente; aceptar lag para análisis de tendencias

Reflexión: ¿Se Eligió la Tecnología Correcta?

Tabla 17 Reflexión sobre la tecnología elegida por módulo

Módulo	Tecnología Actual	¿Correcta?	Análisis
Pedidos (order + order_item + payment)	PostgreSQL	Sí	ACID e integridad referencial son mandatorios para transacciones financieras.
Catálogo de productos (enriquecido)	MongoDB	Sí	Especificaciones variables por categoría, búsqueda de texto, reseñas embebidas.
Sesiones de usuario	MongoDB	Sí	TTL de 24 h, carrito embebido, lecturas de documento único son el caso de uso ideal.

Módulo	Tecnología Actual	¿Correcta?	Análisis
Registros de eventos	MongoDB	Sí	Escrituras de solo-anexar de alto volumen, TTL de 90 días, pipeline de agregación.
Geolocalización	PostgreSQL	Ambos	Dataset pequeño (~19 k filas); funciona en cualquiera; mejor aquí por co-ubicación de JOINS.
Reseñas	PostgreSQL	Posible mejora	Podría migrarse a MongoDB para reducir JOINS en consultas de productos, pero pierde integridad referencial.

Nota. Conclusión: ningún módulo tiene una tecnología claramente incorrecta. La división es arquitectónicamente sólida.

Compensaciones por Escenario Crítico de Negocio

Tabla 18 Compensaciones CAP por escenario crítico de negocio

Escenario	Prioridad CAP	Estrategia Clave
Black Friday (tráfico 10x)	A > C — Disponibilidad primero	Replicación asíncrona, caché Redis, circuit breaker, idempotencia de pedidos
Auditoría Financiera	C > A — Consistencia primero	Aislamiento serializable, replicación síncrona, log de auditoría completo
Lanzamiento de Producto	Dividido — CP pedidos, AP analítica	Pedidos CP en PostgreSQL, analítica AP en MongoDB — cada uno optimizado para su carga

Viabilidad de Arquitecturas de Base de Datos Única

Tabla 19 Viabilidad de arquitecturas de base de datos única

Arquitectura	Puntaje de Viabilidad	Desafíos Clave
100% PostgreSQL	7/10 — Viable pero operativamente complejo	JSONB + GIN para flexibilidad; pg_cron + pg_partman para TTL; mayor sobrecarga de mantenimiento
100% MongoDB	5/10 — Funcionalmente posible pero riesgoso	Transacciones multi-documento para ACID (sobrecarga); sin integridad referencial nativa; riesgo de datos financieros inconsistentes

Lecciones Aprendidas sobre Arquitecturas Híbridas

Fortalezas

- **Separación clara de responsabilidades:** OLTP en PostgreSQL, OLAP en MongoDB — cada motor optimizado para su carga de trabajo.
- **Flexibilidad sin sacrificar integridad:** El catálogo evoluciona libremente en MongoDB mientras los pedidos mantienen ACID estricto en PostgreSQL.
- **TTL como característica diferenciadora:** La expiración automática de datos en MongoDB eliminó una carga operativa significativa.

Debilidades

- **Sincronización entre bases de datos:** `products_catalog` debe sincronizarse con `product` en PostgreSQL. Sin CDC (Debezium), la sincronización es manual y propensa a divergencia.
- **Doble stack operativo:** El equipo debe conocer tanto PostgreSQL como MongoDB — distintas herramientas de monitoreo, respaldo y alertas.

- **Brecha de rendimiento en '\$lookup'**: Las consultas entre colecciones en MongoDB son menos eficientes que los JOINS de PostgreSQL en tablas co-ubicadas.

Recomendaciones Estratégicas

Plan de Escalado 10x

PostgreSQL (3x → 10x carga actual)

Tabla 20 Plan de escalado PostgreSQL: niveles y capacidad estimada

Nivel de Escala	Acciones	Capacidad Estimada
2-3x	Optimizar connection pooling (PgBouncer), aumentar work_mem a 64 MB, agregar vistas materializadas para dashboards	30–50 usuarios concurrentes
5x	Agregar 2 réplicas de lectura asíncronas, separar tráfico OLTP (primario) de OLAP (réplicas), implementar capa de caché Redis	50–100 usuarios, 500 k órdenes
10x	Desplegar Patroni para HA con failover automático, 3+ réplicas de lectura, patrón CQRS, evaluar Citus para PostgreSQL distribuido	200+ usuarios, 1 M+ órdenes

Nota. Hitos clave: shared_buffers 4 GB→8 GB→16 GB; max_connections 100→200→400 con transaction pooling de PgBouncer.

MongoDB (3x → 10x carga actual)**Tabla 21 Plan de escalado MongoDB: niveles y capacidad estimada**

Nivel de Escala	Acciones	Capacidad Estimada
2-3x	Optimizar índices ESR, aumentar replica set a 3 miembros, implementar buffer de eventos del lado del cliente	500 eventos/s
5x	Agregar sharding a event_logs (clave de shard hasheada en _id), 2 shards adicionales, clúster analítico separado	2.000 eventos/s, 50 M documentos
10x	Clúster completamente fragmentado (3+ shards), enrutadores mongos dedicados, zone sharding para event_logs vs. products_catalog	5.000+ eventos/s, 200 M+ documentos

Capa de Caché (Redis)**Tabla 22 Objetivos de caché Redis y estimaciones de memoria**

Objetivo de Caché	TTL	Tasa de Aciertos Objetivo	Memoria Estimada
Catálogo de productos (JSON)	60 s	85%+	200 MB en 10x
Sesiones de usuario	300 s	90%+	500 MB en 10x
Datos geográficos	3600 s	95%+	10 MB (estático)
Búsquedas de pedidos (recientes)	30 s	70%+	100 MB en 10x
Total memoria Redis			~1 GB en 10x

Migración de Nivel Gratuito a Producción

PostgreSQL (Supabase)

Tabla 23 Niveles de Supabase para PostgreSQL

Nivel	Especificaciones	Costo Est.	Cuándo Migrar
Supabase Free	500 MB DB, 5 usuarios, 2 GB bandwidth	\$0/mes	Desarrollo inicial
Supabase Pro	8 GB DB, 50 usuarios, 50 GB bandwidth, PITR	\$25/mes	1.000+ órdenes, equipo de 3-5 desarrolladores
Supabase Team	16 GB DB, límites personalizados, SOC2	\$599/mes	10.000+ órdenes, producción con necesidades de cumplimiento
PostgreSQL auto-alojado	Ilimitado (dependiente del hardware)	\$50-200/mes (VPS)	Escalar más allá de los límites de Supabase

MongoDB

Tabla 24 Niveles de MongoDB Atlas

Nivel	Especificaciones	Costo Est.	Cuándo Migrar
Atlas M0 (Free)	512 MB storage, 100 conexiones	\$0/mes	Desarrollo, prototipado
Atlas M10	2 GB storage, clúster dedicado, 100 conexiones	\$57/mes	10 k+ eventos/día, datos de producción
Atlas M30	8 GB storage, 500 conexiones, BI Connector	\$157/mes	100 k+ eventos/día, necesidades analíticas

Nivel	Especificaciones	Costo Est.	Cuándo Migrar
Atlas M60	32 GB storage, 1000+ conexiones, soporte de sharding	\$672/mes	1 M+ eventos/día, clúster fragmentado

Tabla 25 Stack de producción: componentes y costo mensual estimado

Componente	Nivel	Costo Mensual Est.
Supabase Pro	PostgreSQL 8 GB	\$25
MongoDB Atlas M10	2 GB dedicado	\$57
Redis (Upstash)	1 GB	\$15
Hosting de aplicación (Railway/Render)	2 vCPU, 4 GB RAM	\$20–50
Total producción		~\$117–147/mes

Tecnologías Complementarias

Tabla 26 Casos de uso de Redis como capa de caché

Caso de Uso	TTL	Justificación
Catálogo de productos (desnormalizado)	60 s	Reducir la presión de lectura en MongoDB en un 80% durante picos de navegación
Carrito de sesión de usuario	300 s	Reducir latencia de lectura de sesión de ~5 ms a <1 ms
Búsqueda geográfica	3600 s	Datos estáticos, riesgo de desactualización prácticamente nulo
Contadores de rate limiting	1–60 s	Contadores atómicos en memoria, ideal para Redis

Tabla 27 Tecnologías de búsqueda: requisitos y soluciones recomendadas

Requisito de Búsqueda	Solución	Justificación
Búsqueda de texto completo en productos	MongoDB Atlas Search (integrado)	Sin infraestructura adicional; suficiente para búsqueda en una colección.
Búsqueda facetada o entre colecciones	Elasticsearch	Requerido si la búsqueda abarca pedidos + productos + clientes simultáneamente.
Analítica en tiempo real	Agregaciones Elasticsearch	Si el pipeline de agregación de MongoDB se convierte en un cuello de botella a escala.

Tabla 28 Soluciones de observabilidad recomendadas para Ecommify

Necesidad de Monitoreo	Herramienta	Justificación
Métricas de base de datos (conexiones, hit ratio, lag de replicación)	PostgreSQL exporter + MongoDB Atlas monitoring	Integrado o de código abierto; crítico para planificación de capacidad.
Rendimiento de aplicación (APM, rastreo de consultas)	OpenTelemetry + Grafana Tempo	Alternativa de código abierto a Datadog; suficiente para equipos < 10.
Alertas (pager, Slack)	Grafana Alerting + PagerDuty	Se integra con métricas de Prometheus; umbrales configurables por escenario.

Conclusiones

Evaluación del Logro de Objetivos

Tabla 29 Evaluación del logro de objetivos del proyecto

Objetivo	Estado	Evidencia
Esquema PostgreSQL (9 tablas, 3FN)	Completo	postgresql/schema/01_ddl_ecommify.sql
Colecciones MongoDB (5 con validación)	Completo	mongodb/schema/.json — 5 archivos
Optimización de consultas (índices, particionamiento)	Completo	Mejora promedio 54.8%; mejor caso Q04 con 64.7%
Pruebas de carga	Completo	Escalado lineal 5→10 usuarios; 29.76 qps, 0% error
Pruebas de escalabilidad	Completo	Factor promedio ~18x para crecimiento 50x (100→5.000 órdenes)
Análisis CAP por módulo	Completo	7 CP (PG) + 3 AP (MongoDB)
Análisis comparativo PG vs. MongoDB	Completo	PG: 4 victorias, MongoDB: 4 victorias, Empate: 1
Análisis de compensaciones (3 escenarios)	Completo	Black Friday, Auditoría, Lanzamiento de Producto
Plan de escalado 10x	Completo	Sección G.1
Plan de migración a producción	Completo	Sección G.2 (\$117–147/mes)
Estrategia CI/CD	Completo	Sección G.4

Aprendizajes Clave del Equipo

La arquitectura híbrida es una herramienta, no un objetivo: La división PostgreSQL/MongoDB emergió de los requisitos de carga de trabajo, no de un deseo de persistencia políglota. La tecnología de cada módulo fue elegida basándose en las características de sus datos y patrones de acceso.

Los índices son la optimización de mayor retorno sobre inversión: Los 8 índices especializados en PostgreSQL entregaron una mejora promedio del 54.8% en las 9 consultas optimizadas (Q04: 64.7%, Q10: 59.8%, Q03: 58.5%). El patrón de índice ESR en MongoDB eliminó COLLSCAN completamente para consultas de catálogo, y reorganizar `$match` al inicio del pipeline aportó una mejora adicional del 31% en latencia promedio.

El análisis CAP revela suposiciones implícitas (Brewer, 2000; Gilbert y Lynch, 2002): Clasificar cada módulo como CP o AP forzó decisiones explícitas de compensación. El ejercicio reveló que la plataforma asumía implícitamente CP para todos los módulos, cuando AP era en realidad aceptable (y preferible) para sesiones, eventos y catálogo enriquecido.

La complejidad operativa es el costo oculto de las arquitecturas híbridas: Gestionar dos sistemas de bases de datos duplica la superficie operativa. Para Ecommify los beneficios justifican el costo, pero para aplicaciones más simples, una única base de datos (PostgreSQL con JSONB) sería más pragmática.

La sincronización entre bases de datos es el mayor riesgo a largo plazo: Sin Change Data Capture (Debezium + Kafka) para sincronización automática de PostgreSQL → MongoDB, los datos del catálogo enriquecido pueden divergir del catálogo relacional base. Este es el principal punto de fragilidad de la arquitectura (Debezium, 2024).

Reflexión Final

La arquitectura híbrida de Ecommify está **bien justificada y correctamente implementada**. PostgreSQL domina en integridad transaccional (ACID, restricciones, optimizador de consultas maduro), mientras MongoDB sobresale en flexibilidad analítica (esquema en lectura, índices TTL, escalado horizontal). La evidencia empírica — mejora promedio del 54.8% en las 9 consultas PostgreSQL, un 31% de reducción en latencia del pipeline MongoDB (27.62 ms → 19.06 ms) y escalabilidad sub-lineal $O(n^{0.78})$ hasta 5.000 órdenes sin puntos de quiebre — valida las elecciones tecnológicas.

El conocimiento arquitectónico más importante es que **ninguna configuración única funciona para todos los escenarios**. El Black Friday exige disponibilidad, la auditoría financiera exige consistencia, y el lanzamiento de productos exige un enfoque dividido. El diseño híbrido de Ecommify soporta naturalmente esta configuración basada en escenarios, con cada base de datos optimizada para su carga de trabajo central.

El proyecto demuestra que una arquitectura híbrida bien diseñada puede lograr lo mejor de ambos mundos: la integridad de PostgreSQL y la flexibilidad de MongoDB, cada una aplicada donde agrega más valor. Para el caso de uso de Ecommify — una plataforma de comercio electrónico con patrones de carga de trabajo diversos — este enfoque no solo es viable sino óptimo.

I. Referencias y Anexos

Archivos del Proyecto y sus Propósitos

Tabla 30 Archivos del repositorio y sus propósitos

Archivo	Propósito
postgresql/schema/01_ddl_ecommify.sql	DDL completo: 9 tablas, ENUMs, FK, CHECK, JSONB, particionamiento
postgresql/schema/02_indexes_ecommify.sql	8 índices especializados con análisis ANTES/DESPUÉS
postgresql/schema/03_partition_optimization.sql	Funciones de auto-creación y archivado de particiones
postgresql/queries/base/Q01-Q10.sql	10 implementaciones de consultas base
postgresql/queries/optimized/	9 variantes de consultas optimizadas con mejoras documentadas
mongodb/schema/products_catalog_schema.json	Validación de esquema JSON para catálogo de productos enriquecido
mongodb/schema/user_sessions_schema.json	Esquema JSON para sesiones con definición de índice TTL
mongodb/schema/event_logs_schema.json	Esquema JSON para registros de eventos con TTL
tests/performance/load_test.py	Script de prueba de carga con usuarios concurrentes
tests/performance/scalability_test.py	Script de prueba de escalabilidad con dataset creciente
tests/performance/complex_query_test.py	Comparación de consultas base vs. optimizadas
docs/U6_Evidencias/	Gráficas de rendimiento U6 (fig1–fig6)

Archivo	Propósito
notebooks/U5_PostgreSQL_Optimizacion.ipynb	Notebook de optimización PostgreSQL (EXPLAIN ANALYZE)
notebooks/U5_MongoDB_Optimizacion.ipynb	Notebook de optimización MongoDB (índices ESR, pipeline)

Referencias

Las referencias se presentan en formato APA 7.^a edición (American Psychological Association, 2020). Las entradas se ordenan alfabéticamente por apellido del primer autor o nombre de organización.

American Psychological Association. (2020). Publication manual of the American Psychological Association (7.^a ed.). <https://doi.org/10.1037/0000165-000>

Brewer, E. A. (2000). Towards robust distributed systems [Conferencia magistral]. Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC). Portland, OR. <https://dl.acm.org/doi/10.1145/343477.343502>

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>

Debezium. (2024). Debezium documentation: Change data capture for multiple databases. Red Hat. <https://debezium.io/documentation/>

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM SIGACT News, 33(2), 51–59. <https://doi.org/10.1145/564585.564601>

MongoDB, Inc. (2023a). Aggregation pipeline. MongoDB Documentation.
<https://www.mongodb.com/docs/manual/aggregation/>

MongoDB, Inc. (2023b). Model your data: Equality, Sort, Range (ESR) rule. MongoDB Documentation.
<https://www.mongodb.com/docs/manual/tutorial/equality-sort-range-rule/>

MongoDB, Inc. (2023c). TTL indexes. MongoDB Documentation.
<https://www.mongodb.com/docs/manual/core/index-ttl/>

Olist. (2018). Brazilian E-Commerce Public Dataset by Olist [Conjunto de datos]. Kaggle.
<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

PostgreSQL Global Development Group. (2023a). PostgreSQL 17 documentation: Table partitioning. <https://www.postgresql.org/docs/17/ddl-partitioning.html>

PostgreSQL Global Development Group. (2023b). PostgreSQL 17 documentation: Index types. <https://www.postgresql.org/docs/17/indexes-types.html>

PostgreSQL Global Development Group. (2023c). PostgreSQL 17 documentation: Using EXPLAIN. <https://www.postgresql.org/docs/17/using-explain.html>

Sqitch. (2024). Sqitch: Sensible database change management. <https://sqitch.org/>

Supabase. (2024). Supabase documentation: Architecture.
<https://supabase.com/docs/guides/getting-started/architecture>